



PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS
Instituto de Ciências Exatas e Informática
Departamento de Ciência da Computação

Relatório Projeto 6

Semáforo Inteligente com Arduino Shield Multi-Function

Professores:

Bernardo Cunha Pereira Guerra
Mariana de Oliveira Santos
Marta Dias Moreira Noronha

Integrantes do grupo:

Agnes Vitoria Almeida Fonseca
Ana Carolina Almeida Viana
Ana Luiza Moura dos Santos
Arthur Soares de Carvalho
Camila Tavares Souza

1. Introdução e Problema

Problema escolhido: A ineficiência e a rigidez dos semáforos convencionais, que operam com ciclos fixos independentemente do fluxo de veículos e das condições do ambiente, gerando congestionamentos desnecessários e desperdício de tempo tanto para motoristas quanto para pedestres.

Importância: O tráfego urbano é um dos maiores responsáveis pela perda de produtividade nas cidades, pelo aumento das emissões de poluentes e por acidentes com pedestres. Semáforos mais inteligentes, capazes de adaptar seus ciclos em tempo real, contribuem diretamente para a segurança viária, a fluidez do trânsito e a redução do consumo de combustível causado por paradas desnecessárias.

2. Proposta de Solução

A proposta é desenvolver um semáforo inteligente que adapta seus ciclos automaticamente a três contextos distintos: o período do dia (detectado pelo fotoreistor), a presença ou ausência de veículos (detectada pelo sensor ultrassônico) e a solicitação de travessia por pedestres (via botão do shield). O sistema é configurado pelo usuário antes de entrar em operação, recebendo via monitor serial os tempos de cada sinal. A partir daí, opera de forma autônoma, ajustando os ciclos conforme as condições do ambiente.

O semáforo de veículos e o de pedestres funcionam de forma sincronizada: enquanto o veicular está verde ou amarelo, o pedestre permanece no vermelho; quando o veicular fecha (vermelho), o pedestre abre (verde). Um buzzer progressivo alerta pedestres durante o vermelho veicular, e um display de 7 segmentos exhibe a contagem regressiva do sinal atual.

3. Descrição do Sistema

O sistema gerencia um semáforo inteligente que monitora o fluxo de veículos e a luminosidade ambiente para adaptar seus tempos de sinalização, além de permitir a solicitação de travessia por pedestres.

3.1 Entradas Utilizadas

Entrada/Sensor	Tipo (Digital/Analógico)	Finalidade
Sensor Ultrassônico	Entrada Digital	Detectar a presença e ausência de veículos na via
Fotoreistor (LDR)	Entrada Analógica	Detectar luminosidade ambiente para alternar entre modo diurno e noturno
Botão do Shield	Entrada Digital	Registrar a solicitação de travessia do pedestre.
Potenciômetro do Shield	Entrada Analógica	Calibrar o limiar de luminosidade

3.2 Saídas Utilizadas

Saída/Atuador	Tipo (Digital/Analógico)	Finalidade
LEDs do Shield	Saídas Digitais	Sinalização veicular
LED vermelho (pedestre)	Saída Digital	Sinalização de parada para pedestres.
Buzzer do Shield	Saída Digital	Emitir alertas sonoros progressivos para avisar o motorista que o sinal vermelho está chegando ao fim.
Display de 7 Segmentos	Saída Digita	Exibir a contagem regressiva dos tempos dos sinais.

4. Montagem do Circuito

4.1 Como os Componentes Foram Conectados

O Shield Multi-Function é encaixado diretamente sobre o Arduino Uno, conectando todos os seus recursos internos automaticamente. Os componentes externos são ligados nos pinos passantes do shield da seguinte forma:

- **Sensor ultrassônico HC-SR04:** VCC → 5 V | GND → GND | TRIG → pino digital 9 | ECHO → pino digital 6.
- **Fotoreistor (LDR):** Uma perna ao 5 V; outra perna ao pino analógico A5 e a um resistor de 10 k Ω ligado ao GND.
- **LED vermelho (pedestre):** Positivo → pino digital 5 | Negativo → Resistor 330 Ω → GND.

Os recursos do shield utilizados pelo código (LEDs de veículo nos pinos 11, 12 e 13; buzzer no pino 3; botão de pedestre no pino A1; potenciômetro no pino A0; display de 7 segmentos são acessados exclusivamente via a biblioteca MultiFuncShield, sem necessidade de fiação adicional.

4.2 Detalhes Importantes da Montagem

Divisor de tensão do LDR: é fundamental que o fotoreistor seja conectado em série com um resistor fixo de 10 k Ω entre o GND e o fotoreistor, com o ponto médio ligado ao pino A5. Sem esse resistor de pull-down, o pino analógico flutuaria, tornando as leituras imprevisíveis.

Proteção do LED de pedestre: o resistor de 330 Ω em série com o LED é indispensável para limitar a corrente e evitar danos ao pino de saída digital do Arduino, que suporta no máximo 40 mA por pino.

Conflito de pinos com o shield: os pinos 4, 7 e 8 são utilizados pelo display de 7 segmentos; os pinos 3, 10, 11, 12 e 13 são usados pelos LEDs e pelo buzzer; os pinos A1, A2 e

A3 são os botões do shield. Portanto, os componentes externos foram deliberadamente alocados nos pinos livres: 5, 6, 9 e A5.

Timer1 compartilhado: a biblioteca MultiFuncShield utiliza o Timer1 do ATmega328P para fazer o multiplexamento do display de 7 segmentos e o gerenciamento do buzzer via ISR a 1 kHz. A biblioteca TimerOne é passada para o shield via `MFS.initialize(&Timer1)`, de modo que o Timer1 fica sob controle exclusivo do shield durante toda a execução — nenhuma outra parte do código deve reconfigurar o Timer1 diretamente.

5. Funcionamento do Código

O firmware é organizado em módulos funcionais independentes, cada um responsável por uma parte do comportamento do semáforo. A seguir, cada módulo é descrito em detalhes.

5.1 Configuração Inicial via Monitor Serial

Antes de o semáforo iniciar seus ciclos, o sistema solicita ao operador, via monitor serial, os três tempos que definirão o comportamento diurno: duração do verde, do amarelo e do vermelho (em segundos). Cada valor só é solicitado após o anterior ser confirmado, e o tempo digitado impresso no monitor serial imediatamente após a leitura — sem mensagens desnecessárias antes disso. Essa parte também possibilita a funcionalidade bluetooth no sistema, com o pedido dos valores do tempo sendo solicitado através do celular do usuário.

A partir dos valores informados, o código calcula automaticamente os tempos noturnos: o verde noturno recebe +3 s em relação ao diurno, enquanto o vermelho noturno recebe -3 s (com piso de 1 s para evitar zero). Esses valores são armazenados em variáveis globais separadas, de modo que os dois perfis (diurno e noturno) coexistem na memória durante toda a execução.

5.2 Leitura dos Sensores com Média de Janela 3

Tanto o fotoresistor quanto o sensor ultrassônico são lidos através de funções dedicadas que implementam um filtro de média móvel de janela 3. A cada chamada, a leitura mais antiga é descartada, as demais são deslocadas e a nova leitura ocupa a posição mais recente. A média das três amostras é então calculada e armazenada em variável global.

Esse filtro elimina ruídos pontuais, como reflexos espúrios do ultrassônico ou oscilações momentâneas do LDR, sem introduzir atraso perceptível na resposta do sistema.

5.3 Detecção do Período do Dia (Modo Diurno e Noturno)

A função de verificação do período do dia compara a média filtrada do LDR com um limiar ajustável. Esse limiar é lido continuamente do potenciômetro embutido no shield (poten/A0) e mapeado para uma faixa de sensibilidade configurável, permitindo que o operador calibre o ponto de corte dia/noite sem alterar o código.

Quando a leitura filtrada do LDR fica abaixo do limiar calibrado, o sistema entra em modo noturno e passa a usar os tempos calculados para a noite. Quando a leitura ultrapassa o limiar, retorna ao modo diurno. A troca de modo afeta diretamente a duração dos próximos ciclos do semáforo.

5.4 Detecção de Fluxo de Veículos (Sensor Ultrassônico)

O sensor ultrassônico mede continuamente a distância até o objeto mais próximo na via. Um limiar de distância é definido via potenciômetro do shield: quando a leitura filtrada fica acima desse limiar por mais de 5 segundos consecutivos, interpreta-se que a via está vazia.

Nessa condição, se o semáforo estiver no verde, o sistema força imediatamente a transição para o amarelo e mantém o semáforo nesse estado indefinidamente sinalizando cautela enquanto não houver veículos. Assim que um veículo é detectado (distância abaixo do limiar), o ciclo normal é reiniciado a partir do verde. Essa lógica evita que o sinal verde permaneça aberto sem necessidade, reduzindo o tempo de espera médio dos pedestres.

5.5 Botão de Pedestre

Os botões do shield (pino A1, ativo em LOW com pull-up interno) funcionam como botão de chamada de pedestre. Quando pressionado, a flag botao é ativada, forçando a transição imediata para o amarelo. O ciclo então prossegue normalmente, amarelo e vermelho até o final do vermelho, quando a flag é limpa e o semáforo retorna ao verde.

O evento é registrado pela biblioteca MultiFuncShield através da função MFS.getButton(), que gerencia o debounce e a fila de eventos de botão internamente via ISR, dispensando qualquer lógica de debounce manual no código.

5.6 Máquina de Estados do Semáforo

O comportamento central do semáforo é implementado como uma máquina de estados com três estados: VERDE, AMARELO e VERMELHO. A transição entre estados é controlada por `millis()`, cada estado possui um tempo de duração e, ao esgotar-se ou ao ser ativado por uma das condições especiais (botão ou ausência de fluxo), as funções `cicloVerde()`, `cicloAmarelo()` e `cicloVermelho()` executam a transição.

A cada transição, os LEDs de veículo são atualizados via `digitalWrite()`, o LED de pedestre é atualizado via `digitalWrite()` (ativo em HIGH, sincronizado com o estado oposto ao veicular) e, após o ciclo completo, os contadores de tempo são reiniciados.

5.7 Contagem Regressiva no Display de 7 Segmentos

A cada segundo (controlado por uma variável `tempoAtual`, `tempoAnterior` e `millis()`), o contador de segundos restantes no estado atual é decrementado e o novo valor é enviado ao display de 7 segmentos via `MFS.write(int)`. O display é atualizado a cada iteração do loop, garantindo que o número exibido sempre reflita a contagem atual sem bloqueios.

A contagem regressiva é reiniciada automaticamente a cada mudança de estado, com o valor inicial correspondente ao tempo configurado para o novo estado (diurno ou noturno, conforme a leitura do LDR naquele momento).

5.8 Alerta Sonoro Progressivo (Buzzer)

Durante o estado VERMELHO, o buzzer do shield emite um alerta sonoro progressivo para informar os pedestres sobre o estado do semáforo. O comportamento é dividido em três fases:

- **Melodia de espera:** enquanto restar mais de 5 segundos no vermelho, o buzzer toca bipes em certos intervalos (controlada por `delay()`).
- **Primeiro Alerta:** depois da melodia de espera, o buzzer aumenta a frequência dos bipes para metade do intervalo da melodia de espera.
- **Alerta final:** nos últimos 2 segundos do vermelho, o buzzer toca bipes mais frequentes, igual a metade do intervalo do primeiro alerta.

Ao sair do estado VERMELHO, o buzzer imediatamente desliga.

6. Pseudocódigo

Algoritmo Semáforo Inteligente

Início

Configurações iniciais de pino e bibliotecas

Inicializar componentes.

Variável pronto ← falso Variável botão ← falso

loop Principal

Enquanto (verdadeiro) faça

Fase 1: Configuração inicial de tempos via Serial / bluetooth

se (pronto == falso) Então

Ler os tempos de sinalização (verde, Amarelo, Vermelho)

pronto ← verdadeiro

Fase 2: Execução autônoma do semáforo.

Se não

Calcular média móvel da distância e da luz.

Ler limites de luz pelo potenciômetro

se (botão do pedestre pressionado) Então botão ← verdadeiro

Fim se.

Ajuste automático Dia/Noite.

se (luz atual > limite E estado == noite) então

Carregar tempos padrão (Dia).

Se não se (Luz atual < limite E estado == Dia) Então

Carregar tempos adaptados (Noite: Verde + 3s, Vermelho - 3s)

Fim se.

Controle de tráfego (via vazia)

se (Distância indicar via vazia por mais de 5 segundos) então

Forçar estado amarelo contínuo e apagar display

Ciclo normal ou pedestre

se não

se (botão == falso) então

Executa temporização sequencial: Verde → amarelo → vermelho

se (estado == Vermelho) então controla bipes do buzzer por tempo restante

Fim se

se não

Executa transição rápida (Amarelo → Vermelho) e reseta botão ← Falso

Fim se

Atualizar display com tempo restante a cada segundo

Fim se

Fim se

Fim enquanto

Fim.

7. Conclusão e Viabilidade

O projeto demonstra de forma satisfatória como um semáforo convencional pode ser transformado em um sistema adaptativo com recursos de baixo custo. A integração entre o Arduino Multi-Function Shield, o sensor ultrassônico e o fotoresistor permitiu implementar três modos de operação distintos, diurno, noturno e ausência de fluxo, além da chamada de pedestre, tudo isso em um único firmware modular e bem estruturado.

Durante os testes, o sistema respondeu corretamente às condições simuladas: a transição dia/noite alterou os tempos como esperado, a detecção de via vazia forçou o amarelo dentro do prazo de 5 segundos, o botão de pedestre interrompeu o ciclo imediatamente e o display exibiu a contagem regressiva sem falhas. O buzzer progressivo, embora limitado pela frequência fixa do shield (~1 kHz), cumpriu sua função de alertar pedestres de forma crescente.

As principais limitações identificadas são: a frequência fixa do buzzer, que impede variação real de tom (apenas de ritmo); a dependência de iluminação ambiente estável para a calibração do LDR.

A implementação em ambiente real é totalmente viável para cruzamentos de baixo volume, como saídas de condomínios, travessias escolares ou ciclovias. Melhorias futuras incluem: contagem de veículos por faixa, integração com semáforos vizinhos para criação de ondas verdes e alimentação por painel solar.

Portanto, o projeto cumpre com êxito o objetivo proposto: um semáforo inteligente, seguro e adaptativo.

8. Repositório do Projeto

Código-fonte disponível em:

Link do GitHub:

<https://github.com/arthursoarescarvalho7-cmd/TI6---Semaforo-Inteligente/tree/main>